Student Performance Prediction System

Phase 3 Report: Model Training, Evaluation & Results

Project ID: AIML-5

Name : Arjun Bhandana(Team Leader)

Class Roll Number:31

University Roll Number :2028660

Section and Group:P , G1

Name : Sarthak Gera

Class Roll Number:21

University Roll Number :2028486

Section and Group:P,G1

Name : Arsh Singh

Class Roll Number:32

University Roll Number :2028663

Section and Group:P,G1

TABLE OF CONTENTS

Sr. No. Topic ..... Page No.

1. Problem Description 3

1.1 Dataset Description 3

1.2 Flow Diagram 4

2. Modules for Project Implementation 5

2.1 Modules Used 5

2.2 Platform Used 6

3. Machine Learning Models Used 7

3.1 Regression Models 7

3.2 Classification Models 8

4. Evaluation Metrics Used 9

5. Results and Visualizations 10

5.1 Regression Results (Feature-Selected Features) 10

5.2 Classification Results (Feature-Selected Features) 11

5.3 PCA-Based Model Results 12

5.4 Model Comparison Summary 13

6. Conclusion and Future Scope 14

7. References 15

1. Problem Description

The Student Performance Prediction System is a Python-based machine learning application developed to automate the analysis and prediction of student academic outcomes. It streamlines the process of collecting student data, training predictive models, and generating performance insights while maintaining accuracy and efficiency. Using core concepts of Machine Learning such as data preprocessing, feature selection, regression/classification algorithms, model evaluation, and visualization libraries, this project minimizes manual effort in identifying at-risk students and supports data-driven academic decision-making.

1.1 Dataset Description

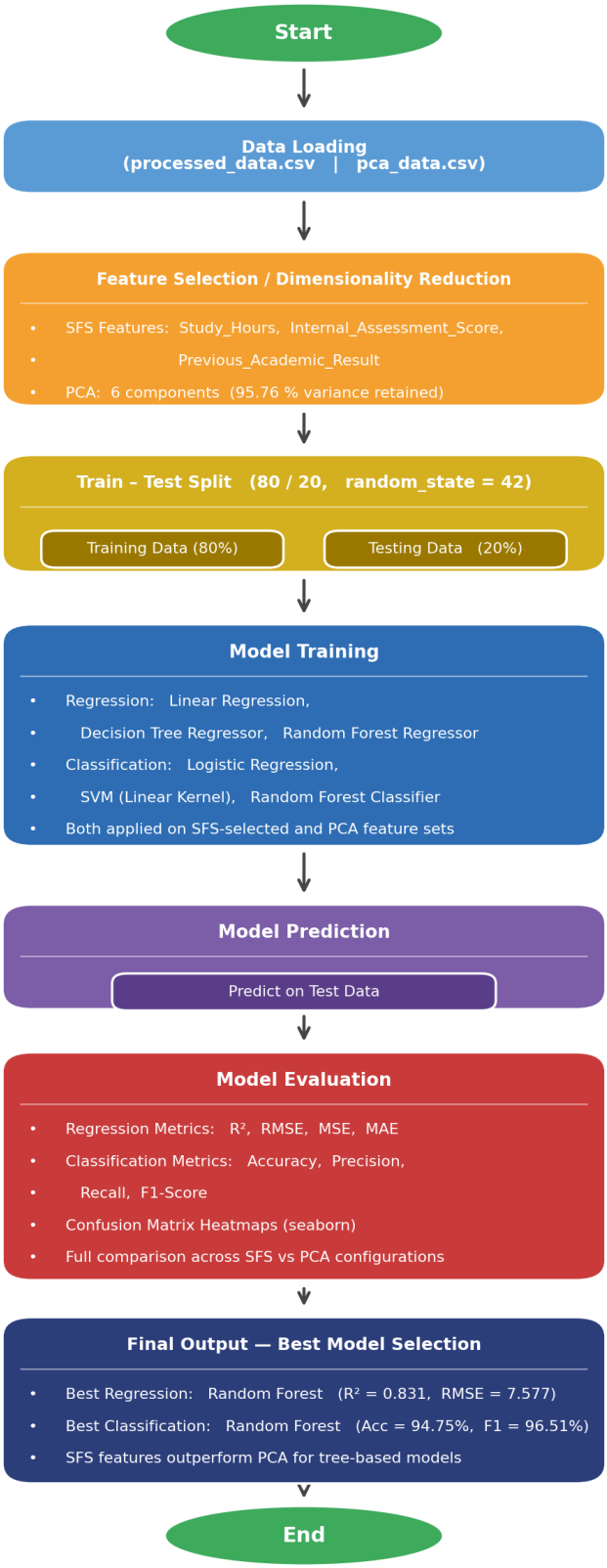
The dataset used is Cleaned_Dataset.csv processed in Phase 1 and further enriched in Phase 2. It contains academic and behavioural features of students along with their final grades. The Phase 2 output, processed_data.csv, was used directly for Phase 3 model training.

Feature	Type	Description
Study_Hours	Float	Average daily study hours per student
Attendance_Percentage	Float	Percentage of classes attended
Internal_Assessment_Score	Float	Marks obtained in internal assessments
Previous_Academic_Result	Float	Academic score from the previous semester
Study_Efficiency	Float	Ratio of output to study hours (engineered)
Internet_Usage_Hours	Float	Daily internet usage in hours
Gender_Male	Binary	Gender encoded: 1 = Male, 0 = Female
Final_Grade	Integer	Target for regression (0–100 scale)
Result	Binary	Target for classification: 1 = Pass, 0 = Fail

1.2 Flow Diagram

The following flow diagram illustrates the end-to-end pipeline of the Student Performance Prediction System from raw data to final predictions:

Raw Data → Data Cleaning (Phase 1) → Feature Engineering & Scaling (Phase 2)
→ Feature Selection (SFS) & PCA → Model Training (Phase 3)
→ Evaluation (R^2, Accuracy, F1, etc.) → Best Model Selection



System Flow Diagram

2. Modules for Project Implementation

2.1 Modules Used

All modules used in Phase 3 are standard Python libraries. The following table lists each module, its purpose, and the specific functions/classes used:

Module	Purpose	Functions / Classes Used
pandas	Data loading and manipulation	pd.read_csv() — reads processed_data.csv and pca_data.csv into DataFrames
numpy	Numerical computation	np.sqrt() — used to compute RMSE from MSE
sklearn.model_selection	Dataset splitting	train_test_split() — splits data into 80% train and 20% test sets
sklearn.linear_model	Regression & Classification	LinearRegression() — predicts Final_Grade; LogisticRegression() — predicts Pass/Fail
sklearn.tree	Decision Tree Regression	DecisionTreeRegressor() — non-linear regression on Final_Grade
sklearn.ensemble	Ensemble Methods	RandomForestRegressor(), RandomForestClassifier() — 100 estimators, random_state=42
sklearn.svm	Support Vector Machine	SVC(kernel='linear') — linear SVM for Pass/Fail classification
sklearn.metrics	Model Evaluation	r2_score, mean_squared_error, mean_absolute_error, accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
seaborn	Visualization	sns.heatmap() — plots confusion matrix as a colour-annotated heatmap
matplotlib.pyplot	Plot Rendering	plt.show(), plt.xlabel(), plt.ylabel(), plt.title()
imblearn	Class Imbalance Handling	SMOTE() — Synthetic Minority Over-sampling applied to training data before all classification models to address class imbalance

2.2 Platform Used

- Jupyter Notebook (Anaconda): Primary development environment for interactive coding, inline output inspection, and visualisation of confusion matrices. All Phase 3 code was written and executed in Jupyter cells.
- Python 3.13 (conda base environment): Runtime used for all computations.
- Visual Studio Code: Used for reviewing and organising code files outside the notebook environment.

- Streamlit: Used to build the deployed web application (Student Performance Predictor) that accepts live student input and returns predicted score and Pass/Fail outcome using the saved score_model.pkl and result_model.pkl.

3. Machine Learning Models Used

Phase 3 trains and evaluates six models — three regression models predicting Final_Grade (a continuous score) and three classification models predicting Result (binary Pass/Fail). Each model is trained on two feature sets: (a) the three SFS-selected features from Phase 2, and (b) the six PCA components from Phase 2.

3.1 Regression Models

a) Linear Regression

Linear Regression models the relationship between the input features and the target (Final_Grade) as a linear combination. It assumes a straight-line relationship between each predictor and the output.

Formula: $\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3$

Implementation: LinearRegression() from scikit-learn, trained on the 3 SFS-selected features (Study_Hours, Internal_Assessment_Score, Previous_Academic_Result) with an 80/20 train-test split.

b) Decision Tree Regressor

A Decision Tree Regressor recursively splits the feature space into regions and assigns the mean target value of training samples in each region as the prediction. It can capture non-linear relationships without feature scaling.

Implementation: DecisionTreeRegressor(random_state=42) from scikit-learn, trained on the same SFS-selected features.

c) Random Forest Regressor

Random Forest is an ensemble of multiple decision trees trained on random subsets of data (bagging) and random subsets of features (feature randomness). The final prediction is the average of all tree predictions, reducing variance and overfitting.

Implementation: RandomForestRegressor(n_estimators=100, random_state=42) — 100 decision trees, each trained on a bootstrapped sample.

3.2 Classification Models

a) Logistic Regression

Logistic Regression uses the sigmoid function to map a linear combination of features to a probability between 0 and 1. A threshold of 0.5 classifies students as Pass (1) or Fail (0).

Formula: $P(y=1) = 1 / (1 + e^{(-z)})$, where $z = \beta_0 + \beta_1 x_1 + ...$

Implementation: LogisticRegression(max_iter=1000) trained on SFS-selected features after applying SMOTE oversampling to handle class imbalance, with a stratified 80/20 train-test split (stratify=y).

b) Support Vector Machine (SVM)

SVM finds the optimal hyperplane that maximises the margin between the two classes (Pass/Fail). A linear kernel was used, which works well when classes are approximately linearly separable in the feature space.

Implementation: SVC(kernel='linear') from scikit-learn, trained with SMOTE-resampled data on a stratified 80/20 split.

c) Random Forest Classifier

Random Forest Classification is an ensemble of decision tree classifiers. Each tree votes for a class label, and the majority vote determines the final prediction. It handles class imbalance and noisy data effectively.

Implementation: RandomForestClassifier(n_estimators=100, random_state=42), trained with SMOTE-resampled data on a stratified 80/20 split.

4. Evaluation Metrics Used

Different metrics are used for regression and classification tasks.

4.1 Regression Metrics

Metric	Formula	Interpretation
R^2 Score (Coefficient of Determination)	$R^2 = 1 - (SS_res / SS_tot)$	Proportion of variance in the target explained by the model. Ranges from 0 to 1; higher is better.
RMSE (Root Mean Squared Error)	$RMSE = \sqrt{(\Sigma(y_i - \hat{y}_i)^2 / n)}$	Average prediction error in the same units as Final_Grade. Lower is better.
MSE (Mean Squared Error)	$MSE = \Sigma(y_i - \hat{y}_i)^2 / n$	Average squared error. More sensitive to outliers than MAE. Lower is better.
MAE (Mean Absolute Error)	$MAE = \Sigma\lvert y_i - \hat{y}_i\rvert / n$	Average absolute error, robust to outliers. Lower is better.

4.2 Classification Metrics

Metric	Formula	Interpretation
Accuracy	Accuracy = (TP + TN) / Total	Fraction of all students correctly classified as Pass or Fail.
Precision	Precision = TP / (TP + FP)	Of students predicted as Pass, how many actually passed. Penalises false positives.
Recall (Sensitivity)	Recall = TP / (TP + FN)	Of students who actually passed, how many were correctly identified. Penalises false negatives.
F1 Score	F1 = 2 × (P × R) / (P + R)	Harmonic mean of Precision and Recall; balances both metrics.
Confusion Matrix	[[TN, FP], [FN, TP]]	2×2 matrix showing actual vs predicted class counts. Visualised as a seaborn heatmap.

5. Results and Visualizations

All models were trained with an 80/20 train-test split (random_state=42). Results are reported on the test set.

5.1 Regression Results — SFS-Selected Features

Features used: Study_Hours, Internal_Assessment_Score, Previous_Academic_Result. Target: Final_Grade.

Model	R^2	RMSE	MSE	MAE
Linear Regression	0.6895	10.268	105.43	7.276
Decision Tree Regressor	0.7180	9.786	95.76	3.641
Random Forest Regressor	0.8310	7.577	57.41	3.248

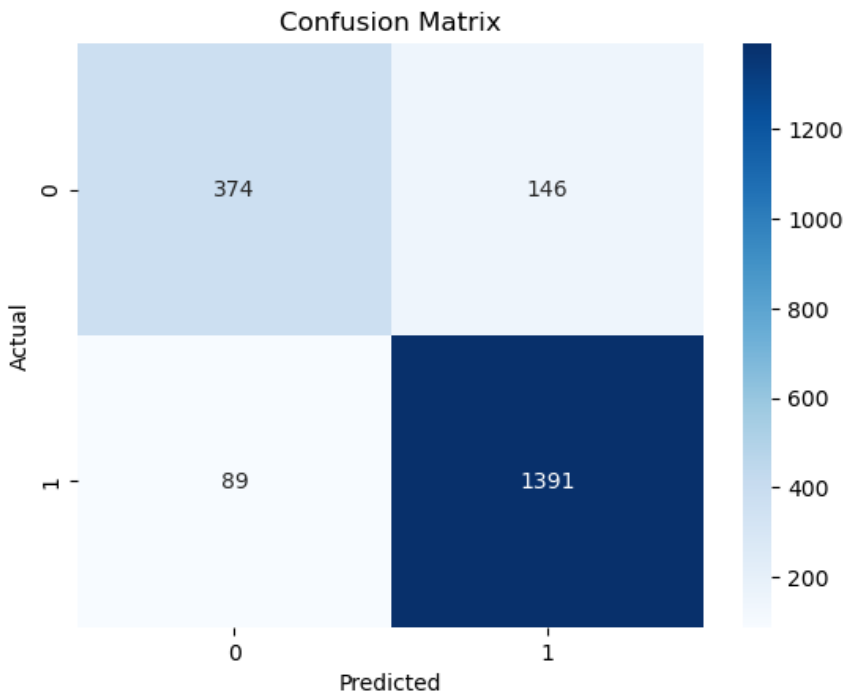
Observation: Random Forest Regressor achieves the highest R^2 of 0.831 and the lowest RMSE (7.577), indicating it explains 83.1% of variance in Final_Grade and has the smallest average prediction error.

5.2 Classification Results — SFS-Selected Features

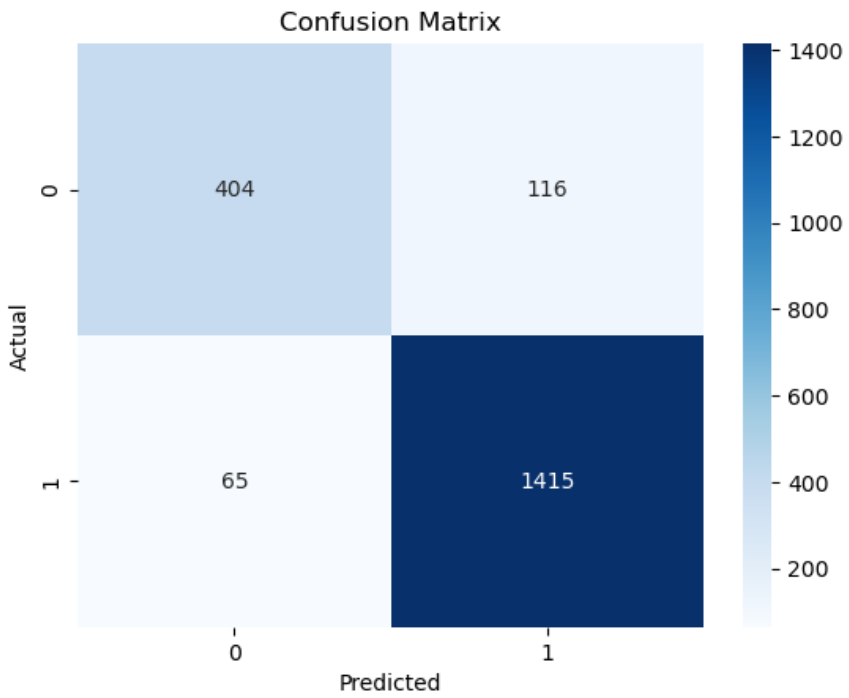
Features used: Study_Hours, Internal_Assessment_Score, Previous_Academic_Result. Target: Result (Pass = 1 / Fail = 0).

Model	Accuracy	Precision	Recall	F1 Score
Logistic Regression	88.25%	90.50%	93.99%	92.21%
SVM (Linear Kernel)	90.95%	92.42%	95.61%	93.99%
Random Forest Classifier	94.75%	95.08%	97.97%	96.51%

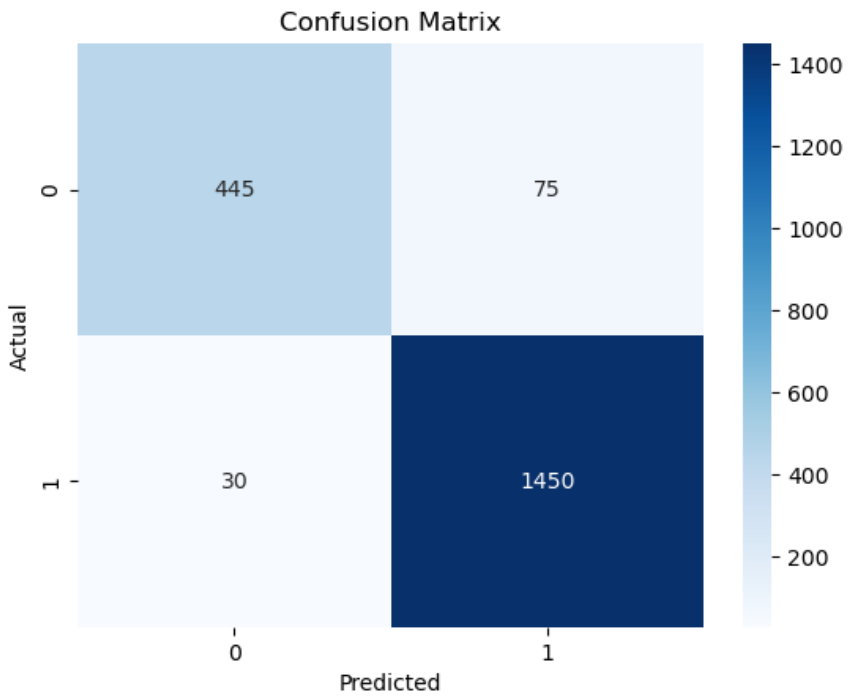
Confusion Matrix — Logistic Regression: [[374, 146], [89, 1391]]
Confusion Matrix — SVM: [[404, 116], [65, 1415]]
Confusion Matrix — Random Forest: [[445, 75], [30, 1450]]



Logistic Regression — Confusion Matrix Heatmap (SFS Features)



SVM — Confusion Matrix Heatmap (SFS Features)



Random Forest — Confusion Matrix Heatmap (SFS Features)

Fig. 2 — Confusion Matrix Heatmaps (Logistic Regression, SVM, Random Forest)

Observation: Random Forest Classifier achieves the best accuracy (94.75%) and F1 Score (96.51%). SVM also performs strongly at 90.95% accuracy. Logistic Regression, while slightly lower, remains competitive at 88.25%.

5.3 PCA-Based Model Results

The same models were retrained using PCA-transformed features (6 components, 95.76% variance retained).

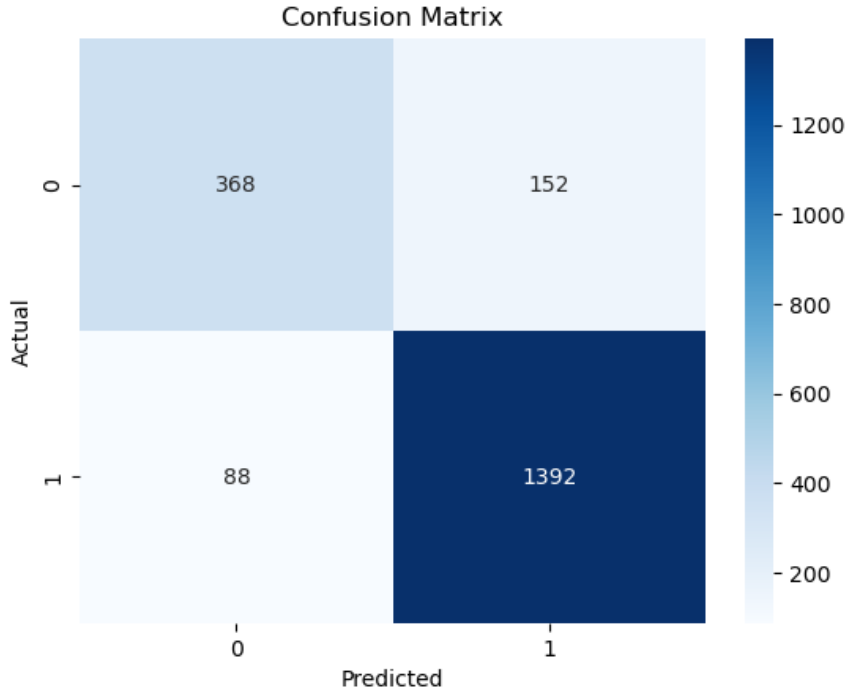
PCA — Regression Results

Model	R^2	RMSE	MSE	MAE
PCA + Linear Regression	0.6902	10.258	105.22	7.335
PCA + Decision Tree Regressor	0.3901	14.392	207.13	8.425
PCA + Random Forest Regressor	0.6768	10.476	109.74	7.366

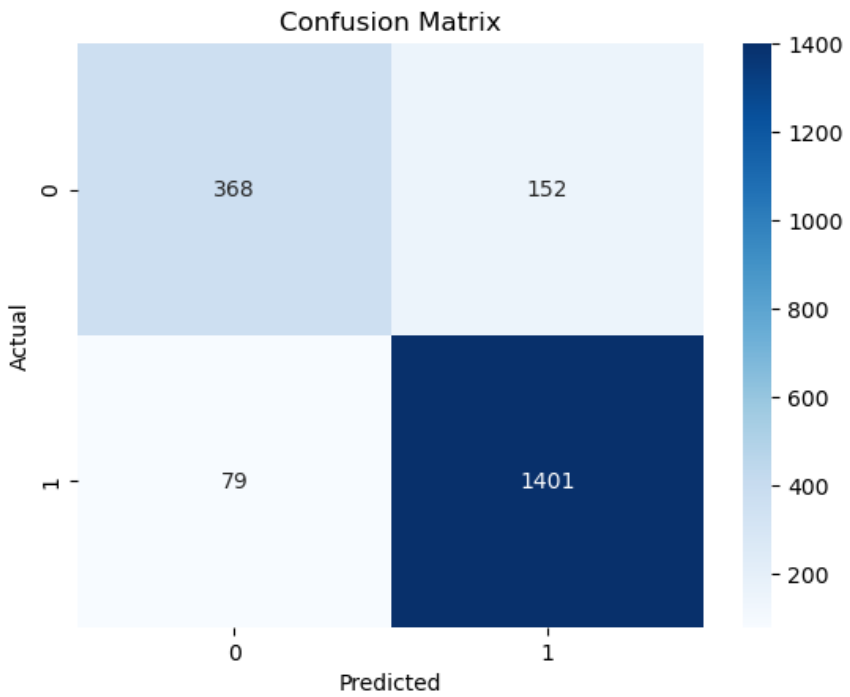
PCA — Classification Results

Model	Accuracy	Precision	Recall	F1 Score
PCA + Logistic Regression	88.00%	90.16%	94.05%	92.06%
PCA + SVM (Linear)	88.45%	90.21%	94.66%	92.38%
PCA + Random Forest Classifier	88.85%	89.26%	96.55%	92.76%

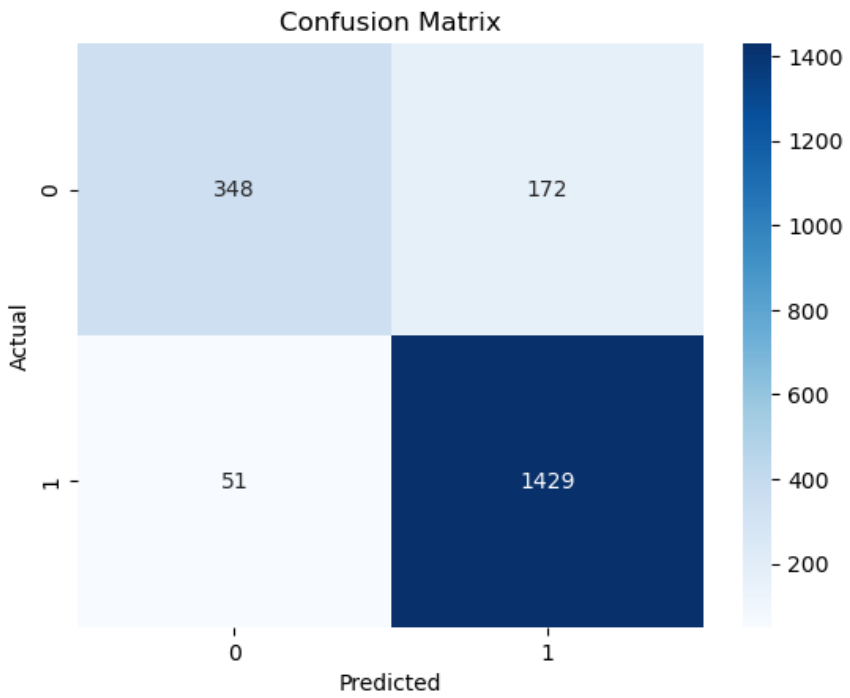
Confusion Matrix — PCA Logistic: [[368, 152], [88, 1392]]
Confusion Matrix — PCA SVM: [[368, 152], [79, 1401]]
Confusion Matrix — PCA Random Forest: [[348, 172], [51, 1429]]



PCA + Logistic Regression — Confusion Matrix Heatmap



PCA + SVM — Confusion Matrix Heatmap



PCA + Random Forest — Confusion Matrix Heatmap

Fig. 3 — PCA-Based Confusion Matrix Heatmaps

5.4 Model Comparison Summary

The table below compares the best metric from each approach to identify the overall best model.

Model	R^2 / Accuracy	RMSE / F1	Features	Best?
Linear Regression	0.6895	10.268	SFS	–
Decision Tree Regressor	0.7180	9.786	SFS	–
Random Forest Regressor	0.8310	7.577	SFS	**✓ Best Regression**
Logistic Regression	88.25%	92.21%	SFS	–
SVM (Linear)	90.95%	93.99%	SFS	–
Random Forest Classifier	94.75%	96.51%	SFS	**✓ Best Classification**
PCA + Linear Regression	0.6902	10.258	PCA	–
PCA + Decision Tree	0.3901	14.392	PCA	–
PCA + Random Forest Regressor	0.6768	10.476	PCA	–
PCA + Logistic Regression	88.00%	92.06%	PCA	–
PCA + SVM	88.45%	92.38%	PCA	–
PCA + Random Forest Classifier	88.85%	92.76%	PCA	–

Finding: SFS-selected features consistently outperform PCA features for Decision Tree and Random Forest models. PCA is competitive for Linear models but causes significant degradation for tree-based methods.

Fig. 4 — Model Comparison Bar Chart (R^2 / Accuracy across all models)

6. Conclusion and Future Scope

6.1 Conclusion

Phase 3 successfully trained and evaluated six machine learning models (three regression, three classification) on the preprocessed student academic dataset. The key findings are:

- For regression (predicting Final_Grade): Random Forest Regressor with SFS-selected features achieves the best performance — $R^2 = 0.831$, RMSE = 7.577, MAE = 3.248. It explains 83.1% of the variance in student grades.
- For classification (predicting Pass/Fail): Random Forest Classifier with SFS-selected features achieves the best performance — Accuracy = 94.75%, F1 Score = 96.51%. Only 30 actual pass students were misclassified as fail.
- SVM with a linear kernel also performed strongly (90.95% accuracy, F1 = 93.99%), making it a viable lightweight alternative.
- PCA-based models performed comparably to SFS models only for Logistic Regression and SVM (within ~0.5%), but degraded significantly for Decision Tree Regression (R^2 dropped from 0.718 to 0.390), confirming that PCA information compression is not equally beneficial for all algorithms.
- The three most predictive features identified in Phase 2 — Study_Hours, Internal_Assessment_Score, and Previous_Academic_Result — proved highly informative across all models.

Overall, the system demonstrates that student academic performance can be predicted with high accuracy using a small set of key academic and behavioural features, supporting early identification of at-risk students.

6.2 Future Scope

- Hyperparameter Tuning: Apply GridSearchCV or RandomizedSearchCV to optimise Random Forest parameters (n_estimators, max_depth, min_samples_split) for further accuracy gains.
- Deep Learning: Explore neural network-based models (e.g., Multi-Layer Perceptron) for potentially better non-linear modelling of student behaviour.
- Real-Time Prediction: A Streamlit web application has been developed and deployed, allowing users to input five student parameters (Study Hours, Attendance, Internal Score, Previous Result, Internet Usage) and receive instant predicted scores and Pass/Fail outcomes. Future work could extend this to a multi-user platform with authentication and database integration.
- Additional Features: Incorporate features like socio-economic background, parental education, extra-curricular participation, and mental health indicators.

- Multi-class Grade Prediction: Extend classification from binary (Pass/Fail) to multi-class (A/B/C/D/F) using the Grade_Label column.
- SHAP / Explainability: Use SHAP values to provide teachers with interpretable explanations of why a student is predicted to fail.

7. References

Books:

- Let Us Python – Yashavant Kanetkar, BPB Publications
- Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow – Aurélien Géron, O'Reilly Media
- Introduction to Machine Learning with Python – Andreas C. Müller & Sarah Guido, O'Reilly Media

Libraries & Official Documentation:

- scikit-learn Documentation — LinearRegression, DecisionTreeRegressor, RandomForestRegressor, SVC, LogisticRegression, train_test_split, evaluation metrics: https://scikit-learn.org/
- pandas Documentation: https://pandas.pydata.org/
- NumPy Documentation: https://numpy.org/
- seaborn Documentation — heatmap: https://seaborn.pydata.org/
- matplotlib Documentation: https://matplotlib.org/

1. imbalanced-learn (imblearn) Documentation — SMOTE: https://imbalanced-learn.org/

- GeeksforGeeks — Random Forest, SVM, Logistic Regression: https://www.geeksforgeeks.org/
- TutorialsPoint — Machine Learning Tutorials: https://www.tutorialspoint.com/
- Stack Overflow — Debugging & Implementation Queries: https://stackoverflow.com
- Towards Data Science — Model Evaluation Best Practices: https://towardsdatascience.com/